

WEEKLY INTERNSHIP REPORT

Week 6 - Ubuntu Migration, NAT/IP Forwarding, and Real Source-IP Logging Phase

Design and Implementation of an IDS/IPS-Based SOC Platform for Detecting and Preventing Broken Access Control Attacks in REST APIs

Student	Bui Tat Dat
Major	Cyber Security
Main target	crAPI vulnerable API laboratory
Monitoring platform	Wazuh Manager, Wazuh Indexer, Wazuh Dashboard, and containerized Wazuh agents
Core engineering focus	Migration from Windows/WSL deployment to Ubuntu VM, Docker NAT path analysis, IP forwarding validation, segmented SOC/management networking, Nginx edge reverse proxy logging, and realistic attacker source-IP preservation
Current status	The previous source-IP attribution blocker has been resolved for web/API detections by placing an edge reverse proxy in front of crAPI. The crAPI VM now uses a separated attack-facing interface and a SOC/management interface, while Wazuh receives alerts that include the real Kali attacker IP 10.10.1.20, HTTP method, URL, and response status code in the same event.
Supervisor	Tran Dai Duong
Organization	Company / Internship Organization

Hanoi, May 28, 2026

Abstract

This weekly report documents the Week 6 engineering progress of the SOC/IDS laboratory for detecting Broken Access Control and related REST API attack behaviors against crAPI. The previous week confirmed that the custom Wazuh rules were valid through regression testing, but live traffic attribution remained limited because the attacker IP address was translated by the Docker and NAT path. As a result, the web access layer could detect suspicious URLs and status codes, but the source IP seen by the application layer did not represent the real Kali attacker host.

During Week 6, the deployment was moved from the Windows/WSL-based crAPI environment to a dedicated Ubuntu virtual machine running native Docker Engine. This change reduced Docker Desktop and WSL port-forwarding ambiguity and made the lab network path easier to control. The Wazuh server remained on the Windows/WSL side, while crAPI and the Wazuh agent containers were deployed inside the Ubuntu VM. The lab network was then refined into two logical segments: the attack-facing network 10.10.1.0/24 and the SOC/management network 172.16.1.0/24. The Wazuh agents were reconfigured to connect to the Wazuh manager through the management address 172.16.1.1 instead of sharing the attacker-facing subnet. After resolving duplicate agent enrollment issues, all relevant agents became active again.

The main technical work of the week focused on NAT visibility, IP forwarding, and logging architecture. A containerized **network-router** was used to perform DNAT and MASQUERADE for the original crAPI routing design and to produce pre-NAT visibility logs. These logs confirmed that the real Kali source IP 10.10.1.20 could be observed before the application-facing NAT path. However, router logs alone were not sufficient for realistic web attack detection because they contained network-flow fields but not the HTTP URL and response status code. To address this limitation, an Nginx edge reverse proxy was inserted in front of **crapi-web**. The proxy listens on the exposed web port and forwards traffic to the internal crAPI web service while writing an access log that contains the original attacker IP, HTTP method, URI, status code, response size, referrer, and user-agent.

After this change, Wazuh successfully generated live dashboard alerts for sensitive path probing, authentication endpoint scanning, and BOLA/IDOR object access. The resulting events showed the real source IP 10.10.1.20, the requested URLs `/.env`, `/identity/api/auth/admin`, and `/api/users/1`, and the expected HTTP status codes. Therefore, the Week 5 source-IP preservation blocker was resolved by combining Ubuntu-native Docker deployment with edge reverse proxy logging.

Keywords: SOC, Wazuh, crAPI, REST API Security, Broken Access Control, BOLA, IDOR, BFLA, NAT, IP Forwarding, Docker Networking, Reverse Proxy, Nginx, Source-IP Preservation, Detection Engineering.

List of Acronyms

Acronym	Meaning
API	Application Programming Interface
BFLA	Broken Function Level Authorization
BOLA	Broken Object Level Authorization
BOPLA	Broken Object Property Level Authorization
DNAT	Destination Network Address Translation
IDS	Intrusion Detection System
IP	Internet Protocol
IDOR	Insecure Direct Object Reference
JWT	JSON Web Token
NAT	Network Address Translation
OSSEC	Open Source Security Event Correlator, the foundation used by Wazuh
SOC	Security Operations Center
SSRF	Server-Side Request Forgery
TCP	Transmission Control Protocol
VM	Virtual Machine
WAF	Web Application Firewall
WSL	Windows Subsystem for Linux

Contents

Abstract	i
List of Acronyms	ii
1 Weekly Progress Overview	1
1.1 Purpose of This Weekly Report	1
1.2 Completed Work This Week	1
1.3 Main Result	2
1.4 Current Project Status	2
2 Background: Week 5 Source-IP Limitation	2
2.1 Observed Limitation Before Week 6	2
2.2 Why Rule Testing Alone Was Not Enough	3
2.3 Week 6 Engineering Objective	3
3 Migration from WSL to Ubuntu VM	3
3.1 Reason for the Migration	3
3.2 Updated Host Roles and Network Segmentation	3
3.3 Wazuh Agent Connectivity	4
4 NAT, IP Forwarding, and Router Visibility	5
4.1 Containerized Router Purpose	5
4.2 Why IP Forwarding Was Required	5
4.3 Router NAT Rules	5
4.4 Pre-NAT Router Logs	5
4.5 Router Log Limitation	6
5 Edge Reverse Proxy Implementation	6
5.1 Reason for Adding the Proxy	6
5.2 Updated Architecture	6
5.3 Proxy Log Format	7
5.4 Port Layout After the Proxy Change	7

5.5	Wazuh Agent Mount Change	7
5.6	Proxy Log Validation	7
6	Wazuh Integration and Detection Validation	8
6.1	Agent Log Collection Configuration	8
6.2	Duplicate Agent Name Resolution	8
6.3	Live Alert Evidence	8
6.4	Interpretation of the Three Alerts	9
6.5	Why This Result Is More Realistic	9
7	Rule Engineering Context for Week 6	10
7.1	No Major Rule Rewrite Was Required	10
7.2	Rules Confirmed by Live Traffic	10
7.3	Router Rule Reassessment	10
8	Technical Challenges and Resolutions	11
8.1	Challenge 1: Access Log Symlinked to Standard Output	11
8.2	Challenge 2: Docker NAT and Application Source IP	11
8.3	Challenge 3: Incorrect Bind Mount	11
8.4	Challenge 4: Duplicate Agent Name	11
9	SOC Analyst Interpretation	12
9.1	How an Analyst Should Read the New Alerts	12
9.2	Example Triage Notes	12
10	Current Limitations	12
10.1	Request Body Visibility	13
10.2	Header Visibility	13
10.3	Router Rule Hardcoding	13
10.4	Log Rotation	13
11	Plan for Week 7	13
11.1	Engineering Tasks	13
11.2	Expected Deliverables for Next Report	14

12 Conclusion	14
A Appendix: Key Commands and Evidence	15
A.1 Proxy Test Requests from Kali	15
A.2 Proxy Access Log Evidence	15
A.3 Wazuh Agent Connectivity and Status Check	15
A.4 Wazuh Alert Search Command	15
A.5 Recommended Wazuh Dashboard Fields	15
B Appendix: Reproducible Nginx Edge Proxy Configuration	17
C Appendix: Weekly Evidence Matrix	18
References	19

List of Figures

1	Week 6 segmented edge proxy and Wazuh management architecture	6
---	---	---

List of Tables

2	Week 6 completed engineering tasks	1
3	Updated lab host roles with network segmentation	4
4	Updated exposed service ports	7
5	Observed live Wazuh alerts after edge proxy deployment	9
6	Rules confirmed by live proxy logs	10
7	SOC triage value of the new proxy-based alerts	12
8	Week 6 evidence matrix	18

1 Weekly Progress Overview

1.1 Purpose of This Weekly Report

This report continues the Week 5 work on Wazuh rule validation and source-IP troubleshooting. In Week 5, the custom rule logic was validated using `wazuh-logtest`, but live dashboard validation was still limited by NAT behavior. The attacker machine was Kali with IP `10.10.1.20`; however, logs visible at the web layer showed a translated Docker or router IP instead of the real attacker IP. This blocked realistic source-IP-based triage and same-source correlation.

The Week 6 purpose was therefore not to rewrite the detection rules. The main purpose was to correct the lab architecture and log collection path so that Wazuh alerts could contain the real attacking host IP together with HTTP attack details.

1.2 Completed Work This Week

The main completed tasks are summarized in Table 2.

Table 2: Week 6 completed engineering tasks

Task	Result
Migration to Ubuntu VM	crAPI was moved from the Windows/WSL Docker Desktop environment to a dedicated Ubuntu VM running native Docker Engine.
Wazuh agent reconfiguration	Wazuh agent containers were reconfigured to connect to the Wazuh manager using the SOC/management address <code>172.16.1.1</code> .
NAT and IP forwarding validation	A containerized router was used to validate DNAT, MASQUERADE, and pre-NAT traffic observation.
Proxy logging design	An Nginx edge reverse proxy was inserted in front of <code>crapi-web</code> to produce realistic web access logs.
Real source-IP validation	Proxy access logs confirmed that Kali <code>10.10.1.20</code> appears in the same log event as HTTP method, URI, status code, and user-agent.
Wazuh live alert validation	Wazuh Dashboard displayed custom bGlobal alerts using the real source IP and attack URL.

1.3 Main Result

Week 6 main result

The previous source-IP limitation was resolved for web/API detection. The edge proxy access log now produces events such as 10.10.1.20 - - [...] "GET /.env HTTP/1.1" 200 Wazuh collects this log through `wazuh-agent-web` and triggers custom rules for sensitive path probing, authentication endpoint scanning, and BO-LA/IDOR object access.

1.4 Current Project Status

The project is now in a stronger state for the final SOC/IDS demonstration. The rule set was already validated in Week 5. In Week 6, the live traffic pipeline was corrected so that the dashboard evidence is more realistic and analyst-friendly. The new architecture supports the following evidence chain. The attack traffic stays on 10.10.1.0/24, while Wazuh agent traffic is forwarded through the SOC/management segment 172.16.1.0/24:

Kali attacker → edge proxy access.log → Wazuh agent → Wazuh Manager →
Dashboard alert

2 Background: Week 5 Source-IP Limitation

2.1 Observed Limitation Before Week 6

Before this week, the lab could generate rule matches, but the live logs did not always preserve the real attacking host IP at the same layer where the HTTP URL and response status were visible. The attacker traffic originated from Kali 10.10.1.20, but web/application logs could show Docker-side addresses, such as 172.20.0.2 or similar bridge addresses. This was technically expected because the request path passed through Docker port publishing, a containerized router, DNAT, and source NAT.

The limitation mattered because many useful SOC triage questions rely on the real source IP:

- Which host attempted the API object access?
- Is the same host scanning many object IDs?
- Is the same host probing multiple authentication endpoints?
- Is the activity coming from an attacker VM, an internal service, or a Docker gateway?

2.2 Why Rule Testing Alone Was Not Enough

The Week 5 `wazuh-logtest` results proved that the XML patterns were valid. However, synthetic test logs do not prove that the live deployment is producing the right fields. A rule can be correct and still fail operationally if the log source does not contain the required fields. For this project, the missing operational property was realistic attribution: source IP, HTTP method, URI, status code, and user-agent needed to be present together in the same web event.

2.3 Week 6 Engineering Objective

The Week 6 objective was to make the live logs behave like a realistic web/API monitoring layer. The desired output format was:

```
10.10.1.20 - - [28/May/2026:15:45:57 +0000] "GET /.env HTTP/1.1" 200 201 "-" "curl/8.13.0"
10.10.1.20 - - [28/May/2026:15:45:57 +0000] "GET /identity/api/auth/admin HTTP/1.1" 404 159 "-" "curl/8.13.0"
10.10.1.20 - - [28/May/2026:15:45:57 +0000] "GET /api/users/1 HTTP/1.1" 404 159 "-" "curl/8.13.0"
```

This format is compatible with the Apache/Nginx-style web access log decoder and allows Wazuh to populate fields such as source IP, HTTP method, URL, and status code.

3 Migration from WSL to Ubuntu VM

3.1 Reason for the Migration

The initial crAPI deployment used Windows, WSL, and Docker Desktop. This deployment was convenient but added network translation layers that made source-IP preservation harder to reason about. For a SOC/IDS demonstration, the lab should be stable and explainable. The project was therefore moved to a dedicated Ubuntu VM so that Docker Engine, container networks, port publishing, and local routing could be controlled more directly.

3.2 Updated Host Roles and Network Segmentation

Table 3 describes the corrected Week 6 host roles after separating the attacker-facing network from the SOC/management network. The Ubuntu crAPI VM is intentionally configured as a dual-homed host: one interface receives attacker traffic, and another interface forwards monitoring traffic to Wazuh.

Table 3: Updated lab host roles with network segmentation

Zone	Host / Interface	Example IP	Role
Attack network	Kali VM	10.10.1.20	Attacker machine used to generate API probing, sensitive path probing, and object-access requests.
Attack network	Ubuntu VM attack interface	10.10.1.130	Exposes the crAPI edge proxy and optional router test ports to the attacker network.
SOC / management network	Ubuntu VM Wazuh-agent interface	172.16.1.128	Provides a dedicated path for Wazuh agent containers to forward logs to the Wazuh manager.
SOC / management network	Windows/WSL Wazuh server	172.16.1.1	Runs the Wazuh single-node server stack, including manager, indexer, and dashboard. This segment should not be directly reachable from the Kali attacker host.

3.3 Wazuh Agent Connectivity

All crAPI-side Wazuh agents were configured to connect to the Wazuh manager through the SOC/management address 172.16.1.1. This replaced the earlier use of the attacker-facing address 10.10.1.1 and makes the monitoring path easier to defend and explain. The Kali attacker only needs to reach the exposed crAPI service on 10.10.1.130; it does not need direct connectivity to the Wazuh server.

The agent configuration uses TCP port 1514 for event forwarding and TCP port 1515 for enrollment. Connectivity was validated from the crAPI VM to 172.16.1.1:1514, 172.16.1.1:1515, and 172.16.1.1:55000. During troubleshooting, the agents encountered duplicate agent-name errors because the manager still retained older registrations with the same names. The fix was to remove the old manager-side entries and recreate the agent containers so they could enroll again through the management network.

The final agent status showed the relevant agents as active:

```
crapi-network-router    Active
crapi-database-layer    Active
crapi-backend-layer     Active
crapi-docker-host       Active
crapi-web-gateway       Active
```

4 NAT, IP Forwarding, and Router Visibility

4.1 Containerized Router Purpose

The `network-router` service remained useful as a network visibility layer. Its purpose was to expose crAPI-related ports, forward traffic into the internal Docker networks, and log traffic before application-side NAT. The router service uses IP forwarding and iptables rules to perform DNAT and MASQUERADE.

4.2 Why IP Forwarding Was Required

IP forwarding allows the router container to forward packets between Docker networks. Without IP forwarding, the container could receive packets but would not correctly route them toward the web or mail service. In the lab, IP forwarding was enabled inside the router container using:

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

This is necessary for the router role, but it does not by itself solve the web access-log source-IP problem. IP forwarding forwards packets; it does not change how the downstream web server logs client information after NAT.

4.3 Router NAT Rules

The router layer used DNAT to forward public service ports to internal service containers. Conceptually, the flow is:

```
External web port 8888 -> network-router -> crapi-web:80
External mail port 8025 -> network-router -> mailhog:8025
```

The router also used MASQUERADE for return-path functionality. This made routing operational, but the application web layer could still see the router or Docker bridge as the client.

4.4 Pre-NAT Router Logs

The router log confirmed that the real attacker IP could be captured before the application-facing NAT path:

```
2026-05-28T15:02:01+0000 prefix=CRAPI_WEB_IN SRC=10.10.1.20 DST=172.20.0.2 PROTO=TCP SPT=36960 DPT=8888 IN=eth1
```

This is useful evidence because it shows that the router layer sees the original source IP. However, the router log is a TCP-flow log. It contains source and destination IP/port fields,

but not the HTTP URL or response status. Therefore, router logs alone are insufficient for detailed web/API attack rule matching.

4.5 Router Log Limitation

Router logs are useful for network visibility and pre-NAT attribution. They are less useful for web attack detection because they do not contain the full HTTP request context. For example, the router can show that 10.10.1.20 connected to port 8888, but it cannot show whether the request was `/.env`, `/api/users/1`, or `/identity/api/auth/admin`. This limitation motivated the edge reverse proxy design.

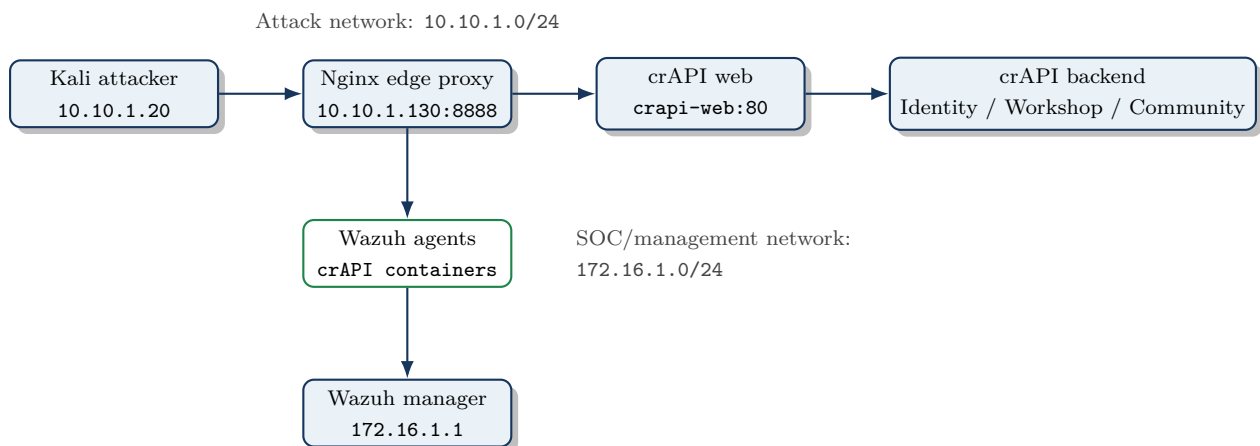
5 Edge Reverse Proxy Implementation

5.1 Reason for Adding the Proxy

The key Week 6 improvement was the insertion of an Nginx edge reverse proxy in front of `crapi-web`. In real environments, web/API detections are often based on logs from a reverse proxy, API gateway, WAF, or load balancer. These layers are closer to the client and can record the original source IP while also recording HTTP method, URI, response status, and user-agent.

5.2 Updated Architecture

Figure 1 summarizes the corrected Week 6 architecture. The attacker path and the Wazuh management path are separated: Kali sends HTTP/API traffic to the crAPI edge proxy on 10.10.1.130:8888, while Wazuh agent traffic is sent to the manager on 172.16.1.1.



Top path: HTTP/API attack traffic and proxy forwarding.
Bottom path: proxy logs forwarded by Wazuh agents over TCP 1514.

Figure 1: Week 6 segmented edge proxy and Wazuh management architecture

5.3 Proxy Log Format

The Nginx proxy was configured to write an access log in a standard web access format:

```
log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                '$status $body_bytes_sent "$http_referer" "$http_user_agent"';
```

This format gives Wazuh the same type of event that it can decode as a web access log. The important difference is that `$remote_addr` now corresponds to the real client IP seen by the edge proxy.

5.4 Port Layout After the Proxy Change

The external port layout was updated so that the proxy became the primary web entry point:

Table 4: Updated exposed service ports

Service path	Port	Purpose
10.10.1.130:8888	Edge proxy web entry point	Main path for realistic web/API logging and Wazuh rule detection.
10.10.1.130:8889	Network-router legacy web path	Optional path for pre-NAT router visibility testing.
10.10.1.130:8026	Network-router Mail-Hog path	Optional mail visibility path after router port shift.

5.5 Wazuh Agent Mount Change

The web Wazuh agent was changed to read the proxy log directory instead of the original `crapi-web` log volume. The intended mapping is:

```
./logs/edge-proxy      -> /var/log/nginx_shared
./wazuh-agent/ossec.conf -> /var/ossec/etc/ossec.conf
```

A temporary error occurred when the proxy log directory was accidentally mounted onto `/var/ossec/etc/ossec.conf`, which is a file path. Docker rejected this with a directory-versus-file bind mount error. The fix was to mount the Wazuh configuration file and the proxy log directory separately.

5.6 Proxy Log Validation

After the fix, the proxy generated the desired log format:

```
10.10.1.20 - - [28/May/2026:15:45:57 +0000] "GET /.env HTTP/1.1" 200 201 "-" "curl/8.13.0"
10.10.1.20 - - [28/May/2026:15:45:57 +0000] "GET /identity/api/auth/admin HTTP/1.1" 404 159 "-" "curl/8.13.0"
10.10.1.20 - - [28/May/2026:15:45:57 +0000] "GET /api/users/1 HTTP/1.1" 404 159 "-" "curl/8.13.0"
```

This output confirms that the detection log now contains both the real source IP and the HTTP attack context.

6 Wazuh Integration and Detection Validation

6.1 Agent Log Collection Configuration

The web agent configuration monitored the edge proxy access log using Apache/Nginx-style parsing:

```
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/nginx_shared/access.log</location>
</localfile>
```

The agent log confirmed that the file was being analyzed:

```
wazuh-logcollector: INFO: (1950): Analyzing file: '/var/log/nginx_shared/access.log'.
wazuh-logcollector: INFO: (1950): Analyzing file: '/var/log/nginx_shared/error.log'.
```

6.2 Duplicate Agent Name Resolution

During the proxy change, the `crapi-web-gateway` agent temporarily became disconnected. Its local log showed:

```
wazuh-agentd: ERROR: Duplicate agent name: crapi-web-gateway. Unable to add agent (from manager)
```

This happened because the Wazuh manager still retained an older registration entry with the same agent name. The old entry was removed on the manager, and the `wazuh-agent-web` container was recreated. After re-enrollment, `crapi-web-gateway` became active again.

6.3 Live Alert Evidence

After sending fresh test requests from Kali to the edge proxy, Wazuh Dashboard showed the expected custom rule alerts. Table 5 summarizes the observed detections.

Table 5: Observed live Wazuh alerts after edge proxy deployment

Rule description	Method	Status	Source IP	URL
Suspicious API/Web Sensitive Path Probing - 2xx/3xx	GET	200	10.10.1.20	/.env
Possible Authentication Endpoint Scanning Against crAPI Identity API	GET	404	10.10.1.20	/identity/api/auth/admin
BOLA/IDOR API Object ID Access Pattern Detected - 4xx	GET	404	10.10.1.20	/api/users/1

6.4 Interpretation of the Three Alerts

Sensitive path probing: The request to /.env returned HTTP 200, meaning the sensitive file path was reachable. This is a strong reconnaissance and exposure indicator.

Authentication endpoint scanning: The request to /identity/api/auth/admin returned HTTP 404. This suggests endpoint guessing against the authentication namespace.

BOLA/IDOR object access: The request to /api/users/1 returned HTTP 404. Even when denied or missing, direct object-ID access attempts are useful signals for BOLA/IDOR probing.

6.5 Why This Result Is More Realistic

The important improvement is that Wazuh now receives a single web event containing all of the following fields:

- real attacker IP: 10.10.1.20;
- HTTP method: GET;
- request URI: /.env, /identity/api/auth/admin, or /api/users/1;
- status code: 200 or 404;
- user-agent: curl/8.13.0;

- custom Wazuh rule description.

This is closer to a production SOC workflow, where detections are often based on reverse proxy, WAF, or API gateway logs rather than raw application logs behind NAT.

7 Rule Engineering Context for Week 6

7.1 No Major Rule Rewrite Was Required

The Week 6 work did not require a major rewrite of the previously validated rule set. The problem was not the rule logic. The problem was that the old log source did not produce realistic source-IP fields. Once the proxy access log was collected, the existing rules could trigger correctly.

7.2 Rules Confirmed by Live Traffic

Table 6 maps the live test requests to the expected rule categories.

Table 6: Rules confirmed by live proxy logs

Rule ID	Detection category	Example URL	Reason for match
110102	Sensitive path probing	<code>/.env</code>	Sensitive path returned <code>2xx/3xx</code> ; in the live test it returned 200.
110204	Authentication end-point scanning	<code>/identity/api/auth/known</code>	authentication-related endpoint returned 404.
110104	BOLA/IDOR object access	<code>/api/users/1</code>	Direct API object ID access returned <code>4xx</code> .

7.3 Router Rule Reassessment

The router rules still provide network visibility, but hardcoding `SRC=10.10.1.20` is lab-specific and less realistic. For future improvements, router rules should parse `SRC=` into `srcip` using a custom decoder, then use correlation logic such as `same_srcip`, `frequency`, and `timeframe`. This would allow the router layer to detect repeated inbound traffic from any source IP rather than only the Kali test address.

8 Technical Challenges and Resolutions

8.1 Challenge 1: Access Log Symlinked to Standard Output

The original `crapi-web` log directory contained symlinks:

```
access.log -> /dev/stdout
error.log  -> /dev/stderr
```

This meant that the Wazuh agent could see an `access.log` path, but the actual request logs were going to Docker stdout rather than a normal shared file. A temporary fix converted the symlink into a regular file in the named volume. The more useful long-term fix was to collect logs from the edge proxy instead.

8.2 Challenge 2: Docker NAT and Application Source IP

When the traffic passed through Docker port publishing and the containerized router, the downstream web server saw a translated address such as `172.19.0.3`. This is expected in a NAT path. The fix was not to force `crapi-web` to see the original IP through NAT, but to introduce an edge proxy layer that receives traffic before the application-side NAT and writes the source IP into the access log.

8.3 Challenge 3: Incorrect Bind Mount

A Docker error occurred when a directory was mistakenly mounted onto the file path `/var/ossec/etc/ossec.conf`. Docker correctly rejected this because the source was a directory and the target was expected to be a file. The mount was corrected so that:

```
./wazuh-agent/ossec.conf -> /var/ossec/etc/ossec.conf
./logs/edge-proxy        -> /var/log/nginx_shared
```

8.4 Challenge 4: Duplicate Agent Name

After recreating the agent containers and changing the manager address to `172.16.1.1`, Wazuh enrollment failed for several agents because matching manager-side entries already existed. The agent logs showed duplicate-name errors for entries such as `crapi-web-gateway`, `crapi-network-router`, `crapi-backend-layer`, `crapi-database-layer`, and `crapi-docker-host`. Removing the old manager-side entries and recreating the containers resolved the issue. This is an important operational lesson: when an agent is removed and recreated with the same name, the Wazuh manager can reject enrollment if the old agent record still exists.

9 SOC Analyst Interpretation

9.1 How an Analyst Should Read the New Alerts

The new dashboard evidence is more useful because it answers the minimum SOC triage questions:

Table 7: SOC triage value of the new proxy-based alerts

Triage question	Field now available in alert
Who generated the suspicious request?	Source IP 10.10.1.20
What API resource was requested?	URL field such as <code>/.env</code> or <code>/api/users/1</code>
What action was attempted?	HTTP method such as <code>GET</code>
Was the request successful?	HTTP status code such as 200 or 404
What tool was used?	User-agent such as <code>curl/8.13.0</code>
What detection logic triggered?	Custom rule description and group

9.2 Example Triage Notes

For the `/.env` alert, the analyst should treat it as high-value reconnaissance because the status code is 200. A successful response implies that a sensitive path returned content. The analyst should check whether the returned content contains credentials, database information, or internal service names.

For the `/identity/api/auth/admin` alert, the analyst should treat it as endpoint discovery. The attacker is guessing authentication-related paths outside the expected login or verification endpoints. If multiple variants appear from the same IP, this should escalate to authentication endpoint scanning.

For the `/api/users/1` alert, the analyst should interpret it as object-level probing. Even when the response is 404, the request pattern indicates that the attacker is testing direct object ID access.

10 Current Limitations

10.1 Request Body Visibility

The current proxy log records URL, method, status, response size, referrer, and user-agent. It does not record request bodies. Therefore, detections for Mass Assignment or some JWT abuse cases remain heuristic if the malicious payload appears only in the body or headers. More accurate detection would require WAF logging, proxy body logging for selected endpoints, or application-layer structured logs.

10.2 Header Visibility

The current access log does not store the full Authorization header. This is good for avoiding accidental credential leakage, but it limits JWT-specific detections. The existing JWT rules can only match indicators that appear in the URL or query string.

10.3 Router Rule Hardcoding

The current router rule set still contains a lab-specific hardcoded Kali IP in one rule. This should be converted to generic correlation logic in a future iteration. The proxy-based web detections no longer require this hardcoded IP because the source IP is decoded from the access log itself.

10.4 Log Rotation

The current proxy log files are stored under `./logs/edge-proxy`. For long-running labs, log rotation should be added to avoid uncontrolled log growth.

11 Plan for Week 7

11.1 Engineering Tasks

The next engineering tasks are:

1. Convert router hardcoded IP detection into generic `same_srcip` correlation.
2. Add test cases for repeated object access to trigger BOLA enumeration correlation.
3. Test additional rule categories with live traffic: SSRF, SQLi/NoSQLi, REST method abuse, BFLA, JWT heuristic, and Mass Assignment heuristic.
4. Prepare dashboard screenshots with stable columns: timestamp, rule description, source IP, method, status, URL, and agent name.

5. Create a SOC triage workflow showing alert review, evidence collection, severity interpretation, and escalation notes.
6. Begin integrating the alert evidence with the incident management workflow, such as IRIS DFIR or TheHive, depending on available time and system resources.

11.2 Expected Deliverables for Next Report

Expected Week 7 deliverables include:

- a live detection matrix covering all main rule categories;
- dashboard screenshots for at least five rule families;
- correlation-rule evidence using repeated requests from the same source IP;
- a concise SOC triage playbook for REST API Broken Access Control alerts;
- a cleaned and reproducible Docker Compose configuration for the final demo.

12 Conclusion

Week 6 resolved the main operational blocker identified in Week 5. The project no longer depends only on synthetic `wazuh-logtest` validation or application logs that hide the real attacker IP behind Docker NAT. By migrating crAPI to an Ubuntu VM, separating the attack network from the SOC/management network, correcting Wazuh agent connectivity, validating NAT and IP forwarding behavior, and adding an Nginx edge reverse proxy, the lab now produces realistic web/API security logs.

The most important result is that Wazuh alerts now show the real Kali attacker IP `10.10.1.20` together with the suspicious URL, method, and response status code. This makes the SOC detection workflow significantly more realistic and defensible. The platform can now demonstrate not only that custom rules match expected patterns, but also that live attacker traffic can be attributed and triaged from Wazuh Dashboard.

A Appendix: Key Commands and Evidence

A.1 Proxy Test Requests from Kali

```
curl -v http://10.10.1.130:8888/.env
curl -v http://10.10.1.130:8888/identity/api/auth/admin
curl -v http://10.10.1.130:8888/api/users/1
```

A.2 Proxy Access Log Evidence

```
10.10.1.20 -- [28/May/2026:15:45:57 +0000] "GET /.env HTTP/1.1" 200 201 "-" "curl/8.13.0"
10.10.1.20 -- [28/May/2026:15:45:57 +0000] "GET /identity/api/auth/admin HTTP/1.1" 404 159 "-" "curl/8.13.0"
10.10.1.20 -- [28/May/2026:15:45:57 +0000] "GET /api/users/1 HTTP/1.1" 404 159 "-" "curl/8.13.0"
```

A.3 Wazuh Agent Connectivity and Status Check

The crAPI VM should reach the Wazuh server through the SOC/management interface:

```
ping -c 4 172.16.1.1
nc -vz 172.16.1.1 1514
nc -vz 172.16.1.1 1515
nc -vz 172.16.1.1 55000
```

The manager-side status can then be checked with:

```
docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/agent_control -l
```

Expected result:

```
crapi-network-router      Active
crapi-database-layer      Active
crapi-backend-layer       Active
crapi-docker-host         Active
crapi-web-gateway         Active
```

A.4 Wazuh Alert Search Command

```
docker exec -it single-node-wazuh.manager-1 sh -lc '
tail -n 1000 /var/ossec/logs/alerts/alerts.log | \
grep -Ei "bGlobal|110102|110104|110204|crapi-web-gateway|10.10.1.20|/.env|auth/admin|/api/users/1" || true
'
```

A.5 Recommended Wazuh Dashboard Fields

For final screenshots, the following fields should be visible:

- @timestamp
- agent.name
- rule.id
- rule.description
- rule.level
- data.srcip
- data.protocol
- data.id or HTTP status field
- data.url
- full_log

B Appendix: Reproducible Nginx Edge Proxy Configuration

```
events {}

http {
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" "$http_user_agent"';

    access_log /var/log/nginx/access.log main;
    error_log /var/log/nginx/error.log warn;

    upstream crapi_backend {
        server crapi-web:80;
        keepalive 32;
    }

    server {
        listen 80;
        server_name _;

        location / {
            proxy_pass http://crapi_backend;
            proxy_http_version 1.1;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_set_header Connection "";
        }
    }
}
```

C Appendix: Weekly Evidence Matrix

Table 8: Week 6 evidence matrix

Evidence item	Observed result	Interpretation
Ubuntu VM deployment	crAPI services run under native Docker Engine on Ubuntu VM	Removes WSL/Docker Desktop ambiguity from the target side.
Wazuh agent manager address	Agents point to 172.16.1.1	Allows Ubuntu-hosted agents to connect to the Wazuh manager through the SOC/management network instead of the attacker-facing network.
Router pre-NAT log	SRC=10.10.1.20 appears in router.log	Confirms the real Kali IP is visible before application-side NAT.
Edge proxy access log	10.10.1.20 appears with URL and status	Provides realistic web/API attack log format.
Web agent logcollector	Agent analyzes /var/log/nginx/shared/access.log	Confirms the Wazuh web agent accesses the proxy log file.
Wazuh agent status	crapi-web-gateway active	Confirms the web agent is connected and can send events.
Dashboard alert for /.env	Sensitive path rule triggered with source IP 10.10.1.20	Confirms successful detection of sensitive path probing.
Dashboard alert for auth path	Authentication endpoint scanning rule triggered	Confirms auth namespace probing detection.
Dashboard alert for /api/users/1	BOLA/IDOR object access rule triggered	Confirms object-level access probing detection.

References

1. Wazuh Documentation, Ruleset and custom rules reference.
2. Wazuh Documentation, Agent enrollment and manager-agent communication.
3. Docker Documentation, Bridge networks and port publishing behavior.
4. Nginx Documentation, Reverse proxy configuration and access log format.
5. OWASP API Security Top 10, Broken Object Level Authorization and Broken Function Level Authorization categories.
6. MITRE ATT&CK, T1190 Exploit Public-Facing Application and T1595 Active Scanning.